Gestion des services

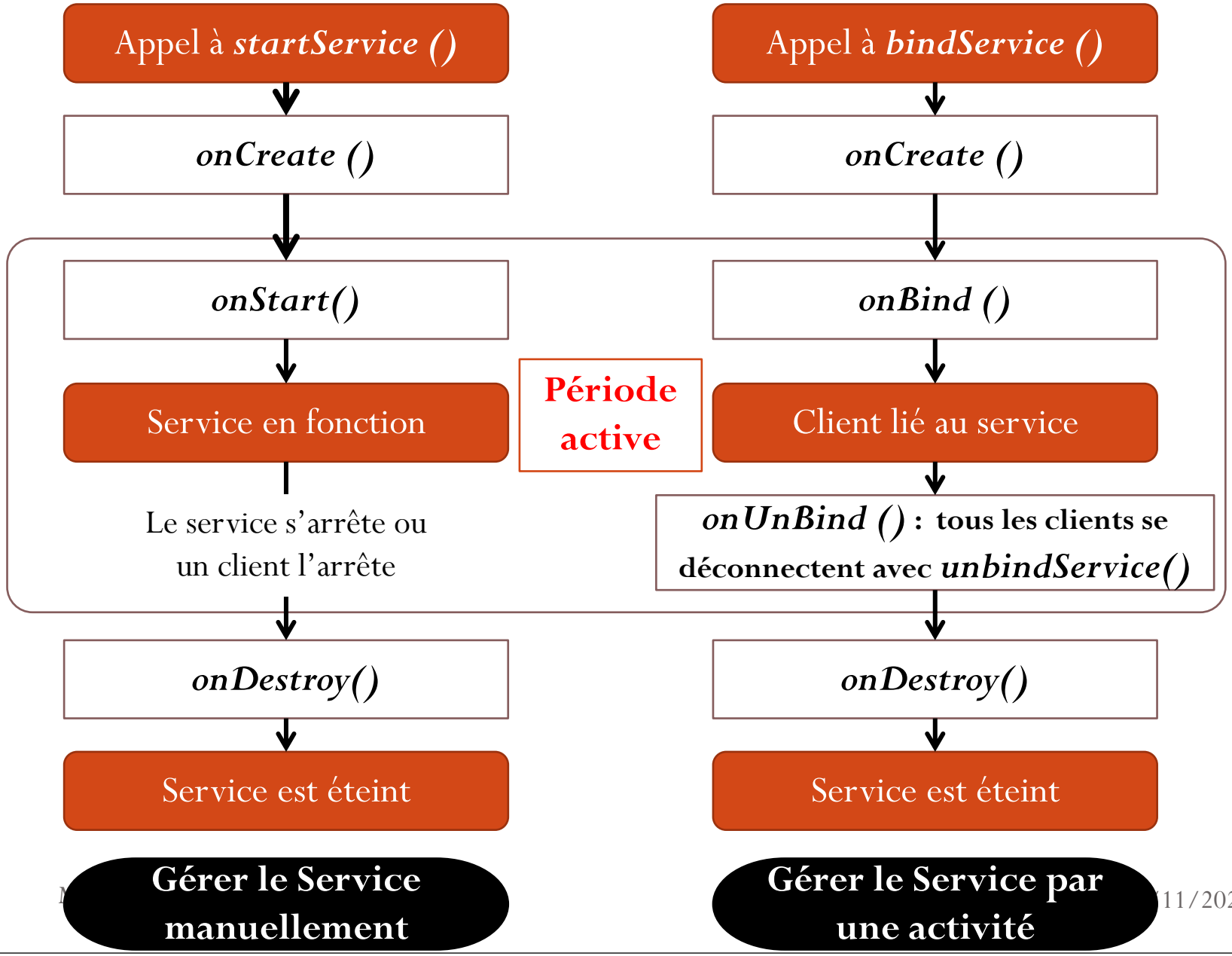
- Service
  - Les services sont des composants applicatifs Android qui s'exécutent en arrière plan (invisibles pour l'utilisateur)
  - Ils permettent d'effectuer des tâches de longue durée ou des opérations qui ne nécessitent pas d'interaction directe avec l'utilisateur (télécharger des fichiers, jouer de la musique en arrière-plan, synchroniser des données avec un serveur)
  - Les services peuvent être contrôlés **manuellement** par l'utilisateur (cas rare) ou généralement contrôlés **par des activités**
  - Il est courant de créer des applications composées d'activités et de services. Dans ce cas, les activités permettent à l'utilisateur de contrôler les services
  - Un service correspond à une classe qui doit hériter de la classe mère « ***Service*** » ou de l'une de ses sous-classes

- Types des services
  - **Service de démarrage** : Il est démarré par un appel à la méthode « ***startService*** » et s'exécute indéfiniment jusqu'à ce qu'il soit arrêté explicitement ou qu'une autre composante appelle la méthode « ***stopService*** » (lecture de musique en arrière-plan)
  - **Service lié** : Il est lié à une activité ou à un autre composant via la méthode de liaison « ***bindService*** » et il continu à fonctionner tant que les composants sont liés à lui (Il s'arrête lorsque le composant auquel il est lié est détruit) (connexion à une base de données locale)
  - **Intentservice** : Une sous-classe de Service optimisée pour les tâches de courte durée. Elle s'arrête automatiquement lorsqu'elle a terminé son travail (télécharger un fichier en arrière-plan)

- Utilisation des services
  - **Tâches de longue durée** : Les services sont idéaux pour les opérations qui peuvent prendre du temps, comme le téléchargement de gros fichiers ou la lecture de musique en arrière-plan
  - **Exécution en arrière-plan** : Les services continuent de s'exécuter même si l'application passe en arrière-plan ou si l'utilisateur passe à une autre application
  - **Notifications** : Les services peuvent envoyer des notifications à l'utilisateur pour l'informer de l'avancement d'une tâche ou d'un événement
  - **Contrôle des médias** : Les services sont souvent utilisés pour contrôler la lecture de médias, comme la musique ou les podcasts

- Cycle de vie d'un service
  - Les services possèdent un cycle de vie comme les activités. Ce cycle de vie est composé de trois principaux états :
    - ***Started***: Le service a été démarré et s'exécute en arrière-plan,
    - ***Bound***: Le service est lié à un composant,
    - ***Destroyed***: Le service a été arrêté.

- Cycle de vie d'un service



- Création d'un service
  - Pour créer un service, il faut hériter de la classe « ***android.app.Service*** »

```
public class MonService extends Service {
    @Override
    public void onCreate() {
        // Placez ici le code qui sera exécuté lors de la création de ce service
    }
    @Override
    public void onStart(Intent intent, int startId) {
        // Placez ici le code qui sera exécuté à chaque démarrage du service
    }
    @Override
    public void onDestroy() {
        // Placez ici le code qui sera exécuté lors de la destruction de ce service
    }
    @Override
    public IBinder onBind(Intent arg0) {
        return null;
    }
}
```

- Création d'un service
  - Les méthodes « ***onCreate*** », « ***onStart*** » et « ***onDestroy*** » sont respectivement appelées lors de la création, le démarrage et de la destruction du service
  - La méthode « ***onBind*** » retourne un objet « ***IBinder*** » qui assure le fonctionnement du mécanisme de communication interprocessus « ***IPC*** » : communication entre le processus qui a appelé le service et le processus dans lequel s'exécute le service

  - Comme les activités, chaque service doit être déclaré dans le fichier de configuration « *manifest* » de l'application :

```
<service android:name=".MonService">
```

- Démarrage et arrêt d'un service
  - Les services peuvent être contrôler **manuellement**
  - Pour démarrer un service **manuellement**, utilisez la méthode « ***startService*** »

```
// Démarre le service manuellement
startService(new Intent(this, MonService.class));
```

  - Pour arrêter un service **manuellement**, utilisez la méthode « ***stopService*** »

```
//Arrêter le service manuellement
stopService(new Intent(this, MonService.class));
```

- Contrôle d'un service depuis une activité
  - Dans la plupart des cas, les services sont contrôlés par des activités
  - Lorsqu'une activité est liée à un service, elle possède une référence permanente vers ce service
  - Pour permettre à une activité de s'associer à un service, la méthode « ***onBind*** » doit être implémentée au niveau du service. Cette méthode retourne un objet de type « ***Ibinder*** » qui sera utilisé plus tard par l'activité pour récupérer la référence permanente vers le service

- Contrôle d'un service depuis une activité
  - La méthode « ***onBind*** » doit renvoyer un objet de type « ***Ibinder*** » propre à votre service, ce qui signifie que vous devez créer votre propre implémentation de l'interface « ***Ibinder*** »
  - Exemple :

```
//Implémentation de l'interface « Ibinder »
public class  MonServiceBinder  extends  Binder {
        MonService  getService() {
            return  MonService.this;
        }
}
private  final  IBinder  mBinder = new MonServiceBinder(); //instance à
//retourner par la méthode onBind
// Redifinition de la méthode onBind
public  IBinder  onBind (Intent intent) {
        return mBinder;
}
```

- Contrôle d'un service depuis une activité
  - Une fois la méthode « ***onBind*** » implémentée au niveau de votre service, on doit connecter l'activité au service à travers une connexion de type « ***ServiceConnection*** »
  - La gestion de la connexion au service au niveau de l'activité se fait en redéfinissant les méthodes « ***onServiceConnected*** » et « ***onServiceDisconnected*** » de la classe « ***ServiceConnection*** » : La première méthode sera appelée lors de la connexion de l'activité au service, la seconde lors de la déconnexion

- Contrôle d'un service depuis une activité
  - Exemple de redéfinition des méthodes de la classe « ***ServiceConnection*** »

```
private  MonService  monService;
// L'objet « ServiceConnection » gère la connexion entre l'activité et le service
private ServiceConnection  maConnexion = new ServiceConnection ( ) {
   // Méthode appelée lorsque la connexion est établie. Récupère la référence
   // vers le service associé
   public void onServiceConnected(ComponentName  classN, IBinder service) {
        monService = ((MonService.MonServiceBinder)service).getService();
   }
   // Méthode appelée lorsque la connexion est rompue
   public void onServiceDisconnected(ComponentName className) {
         monService = null;
   }
};
```

- Contrôle d'un service depuis une activité
  - Une fois que la méthode « ***onBind*** » du service est implémentée et qu'un objet « ***ServiceConnection*** » est prêt, l'activité peut s'associer au service et le lancer à l'aide de la méthode « ***bindService*** ». Cette méthode prend trois paramètres :
    - L'objet « ***Intent*** » du service que vous souhaitez invoquer
    - Une instance de la connexion créée vers le service
    - Un drapeau spécifiant les options d'association, souvent la valeur « ***0*** » ou la constante « ***BIND_AUTO_CREATE*** »
  - Pour déconnecter l'activité du service, on utilise la méthode « ***unBindService*** »

- Contrôle d'un service depuis une activité
  - Exemple d'association du service à l'activité :

```
//méthode onCreate de l'activité
public void onCreate (Bundle bundle) {
    super.onCreate(bundle);
    // Association du service
    Intent  intentAssociation = new Intent(MonActivite.this,
MonService.class);
    bindService (intentAssociation, maConnexion,
Context.BIND_AUTO_CREATE);
}
```

  - Exemple de suppression de l'association entre le service et l'activité :

```
unbindService (maConnexion);
```

MERCI DE VOTRE ATTENTION